

I/O Parallelism

1 Overview

In its simplest form, **I/O parallelism** refers to reducing the time required to retrieve relations from disk by partitioning them over multiple disks. The most common form is **horizontal partitioning** (or declustering), where tuples of a relation are divided among many disks so that each tuple resides on exactly one disk.

2 Partitioning Techniques

Assume there are n disks: $D_0, D_1, \dots, D_{n-1}$.

- **Round-robin:** Tuples are scanned in any order and the i -th tuple is sent to disk $D_{i \bmod n}$. This ensures an even distribution but makes point/range queries complex as all disks must be searched.
- **Hash Partitioning:** Designates one or more attributes as *partitioning attributes*. A hash function h with range $[0, n - 1]$ is applied to the attribute value. If $h(v) = i$, the tuple goes to disk D_i .
- **Range Partitioning:** Distributes tuples by assigning contiguous attribute-value ranges to each disk. A partitioning vector $[v_0, v_1, \dots, v_{n-2}]$ is chosen such that:

- $x < v_0 \rightarrow D_0$
- $v_i \leq x < v_{i+1} \rightarrow D_{i+1}$
- $x \geq v_{n-2} \rightarrow D_{n-1}$

Range Partitioning Example

With 3 disks and vector $[5, 40]$:

- Values < 5 : Disk 0
- Values $5 \leq x < 40$: Disk 1
- Values ≥ 40 : Disk 2

3 Detailed Comparison of Techniques

The efficiency of partitioning depends on the access pattern of the query.

- **Round-robin:**

- *Pros*: Ideal for sequential scans; ensures perfectly balanced data distribution.
- *Cons*: Highly inefficient for point and range queries as every disk must be activated, incurring high startup overhead.
- **Hash Partitioning**:
 - *Pros*: Best for **point queries** on the partitioning attribute (localized to one disk). Good for sequential scans as it balances tuples across disks if the hash function is well-chosen.
 - *Cons*: Poor for range queries (hash functions do not preserve order). Inefficient for point queries on non-partitioning attributes.
- **Range Partitioning**:
 - *Pros*: Best for both **point and range queries** on the partitioning attribute. It provides **data clustering**, narrowing the search to only the relevant disks.
 - *Cons*: Can lead to **execution skew** (hot spots). If many tuples fall into a single range, a single disk handles all the load while others stay idle, wasting parallelism.

4 Comparative Analysis

Partitioning techniques are evaluated based on three access patterns:

1. **Full Scan**: Reading the entire relation.
2. **Point Query**: Locating a tuple associatively (e.g., salary = 50000).
3. **Range Query**: Locating tuples within a range (e.g., 10k < salary < 20k).

Technique	Full Scan		Point Query		Range Query	
Round-robin	Excellent I/O)	(Parallel	Poor (Must scan all disks)		Poor (Must scan all disks)	
Hash	Excellent I/O)	(Parallel	Best (Search one disk)		Poor (No proximity)	
Range	Excellent I/O)	(Parallel	Best (Search one disk)		Best (Search few disks)	

Table 1: Comparison of Access Efficiency

Rule of Thumb for Partitioning

- **Round-robin**: Best for uniform sequential scanning.
- **Hash Partitioning**: Use for equality predicates (point queries).
- **Range Partitioning**: Use for ordered access and range-based predicates.

5 Partitioning a Relation across Disks

To optimize resource usage, the number of disks used should depend on the relation size:

The min(m , n) Rule

If a relation consists of m disk blocks and there are n disks available:

- Very small relations should be assigned to a **single disk**.
- Larger relations should be partitioned across $\min(m, n)$ disks.

Example: If a relation has $m = 3$ blocks and there are $n = 10$ disks, allocate only 3 disks to prevent unnecessary startup overhead.

6 Handling of Skew

Skew refers to the uneven distribution of tuples/work.

Skew Categories

- **Attribute-value skew:** Some values appear frequently in the data (e.g., many employees in a 'Sales' department). All these end up in one partition.
- **Partition skew:** Occurs when the partitioning logic (e.g., range boundaries) is poorly defined, even if the data itself is uniform.

6.1 Constructing Balanced Range Vectors

To create a balanced vector for range partitioning:

1. **Sort** the relation on the partitioning attribute.
2. **Scan** the sorted data.
3. After every $1/n$ -th of the relation, add the current value to the **partitioning vector**.

6.2 Histogram-Based Balancing

Histograms (frequency tables) are used in practice to avoid the cost of sorting.

- **Method:** Consult the histogram to pick ranges such that each range sum is equal to $TotalTuples/n$.
- **Sampling:** If a histogram isn't available, the system can sample random blocks to estimate the distribution.

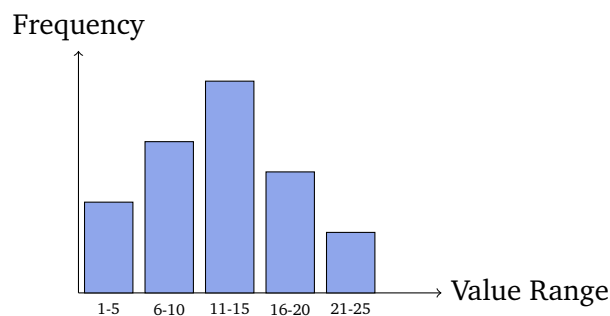


Figure 1: Attribute Value Histogram for range balancing

6.3 Virtual Processor Partitioning

This is a powerful technique to handle skew elegantly:

The Virtual Processor Approach

Instead of partitioning directly to N real processors, we create a much larger number of **virtual partitions** (typically $10\times$ to $20\times$ more than physical ones).

1. **Mapping:** Tuples are mapped to these many small virtual partitions.
2. **Distribution:** Virtual processors are mapped to real physical processors (usually via **round-robin**).

The Logic: If one "normal" range had too much data, it gets split into several smaller virtual ranges. Since these virtual ranges are distributed round-robin among all real processors, the "extra" work is shared across the entire system rather than hitting a single disk/CPU.

Hybrid Approaches

Modern large-scale systems often combine hash and range partitioning via **composite schemes** (e.g., range partitioning for coarse-grained distribution and hash partitioning for fine-grained balancing within ranges).

Through careful partitioning and skew mitigation, parallel databases achieve near-linear speedup in data processing.